

```

import { useState, useEffect } from "react";

const C = {
  void: "#050508", deep: "#0a0b0f", titanium: "#12141a",
  chrome: "#8a9aaa", chromeDim: "#2a3040",
  pearl: "#d4e8f0", pearlDim: "#7ba8bc",
  accent: "#4dd9ff", accentDim: "rgba(77,217,255,0.08)",
  gold: "#c8a84b", goldDim: "#3a2e10", goldFaint: "rgba(200,168,75,0.08)",
  red: "#ff4455", redFaint: "rgba(255,68,85,0.08)",
  green: "#44ffaa", greenFaint: "rgba(68,255,170,0.06)",
  white: "#f0f4f8",
};

// — Fingerprint —————
function generateFingerprint(str) {
  let h1 = 0xdeadbeef, h2 = 0x41c6ce57;
  for (let i = 0; i < str.length; i++) {
    const ch = str.charCodeAt(i);
    h1 = Math.imul(h1 ^ ch, 2654435761);
    h2 = Math.imul(h2 ^ ch, 1597334677);
  }
  h1 = Math.imul(h1 ^ (h1 >>> 16), 2246822507) ^ Math.imul(h2 ^ (h2 >>> 13), 3266);
  h2 = Math.imul(h2 ^ (h2 >>> 16), 2246822507) ^ Math.imul(h1 ^ (h1 >>> 13), 3266);
  const hex = (4294967296 * (2097151 & h2) + (h1 >>> 0)).toString(16).toUpperCase;
  return hex.padStart(12, '0').match(/.{1,4}/g).join('-');
}

// — QR Matrix —————
function generateQRMatrix(data) {
  const size = 21;
  const matrix = Array(size).fill(null).map(() => Array(size).fill(0));
  const addFinder = (row, col) => {
    for (let r = 0; r < 7; r++) for (let c = 0; c < 7; c++) {
      if (r === 0 || r === 6 || c === 0 || c === 6 || (r >= 2 && r <= 4 && c >= 2)
        if (row + r < size && col + c < size) matrix[row + r][col + c] = 1;
    }
  };
  addFinder(0, 0); addFinder(0, 14); addFinder(14, 0);
  for (let i = 8; i < 13; i++) { matrix[6][i] = i % 2 === 0 ? 1 : 0; matrix[i][6]
  let hash = 0;
  for (let i = 0; i < data.length; i++) { hash = ((hash << 5) - hash) + data.charCodeAt(i);
  let s = Math.abs(hash);
  const rand = () => { s = (s * 1664525 + 1013904223) & 0xffffffff; return (s >>> 16) % 255;
  for (let r = 0; r < size; r++) for (let c = 0; c < size; c++)

```

```

    if (matrix[r][c] === 0 && !(r < 8 && c < 8) && !(r < 8 && c > 12) && !(r > 12
        matrix[r][c] = rand() > 0.5 ? 1 : 0;
    return matrix;
}

```

```

function applyArtifactFracture(matrix, sessionId) {
    let seed = 0;
    for (let i = 0; i < sessionId.length; i++) { seed = ((seed << 5) - seed) + sess
    const x = 8 + (Math.abs(seed) % 6);
    const y = 8 + (Math.abs(seed * 31) % 6);
    const f = matrix.map(row => [...row]);
    if (f[y]?.[x] !== undefined) f[y][x] = f[y][x] === 1 ? 2 : 1;
    return f;
}

```

```

function QRDisplay({ data, isArtifact, sessionId, size = 80 }) {
    const matrix = generateQRMatrix(data);
    const display = isArtifact ? applyArtifactFracture(matrix, sessionId) : matrix;
    const cell = size / display.length;
    return (
        <svg width={size} height={size} style={{ display: 'block' }}>
            <rect width={size} height={size} fill="white" />
            {display.map((row, r) => row.map((cell_val, c) => {
                if (!cell_val) return null;
                return <rect key={`-${r}-${c}`} x={c * cell} y={r * cell} width={cell} hei
                    fill={cell_val === 2 ? '#8a6a1a' : '#050508'} />;
            ))}
            {isArtifact && <rect x={0} y={size - 2} width={size} height={2} fill={C.gol
        </svg>
    );
}

```

```

// — Auth Status Badge —————
function AuthBadge({ auth }) {
    const cfg = {
        confirmed: { color: C.green, bg: C.greenFaint, label: '✓ CONFIRMED' },
        pending: { color: C.gold, bg: C.goldFaint, label: '⌚ PENDING' },
        failed: { color: C.red, bg: C.redFaint, label: '✗ FAILED' },
    }[auth.status] || { color: C.chromeDim, bg: C.titanium, label: '— UNSET' };

    return (
        <div style={{
            display: 'inline-flex', alignItems: 'center', gap: 6,
            background: cfg.bg, border: `1px solid ${cfg.color}`,
            borderRadius: 3, padding: '4px 10px',
        }}>
            <div style={{ width: 6, height: 6, borderRadius: '50%', background: cfg.col

```

```

        <span style={{ fontSize: 7, color: cfg.color, letterSpacing: 2, fontFamily:
            {cfg.label}}
        </span>
        {auth.lockedForRelease && (
            <span style={{ fontSize: 7, color: C.gold, letterSpacing: 2, marginLeft:
            }}
        </div>
    );
}

```

```

// — JSON Export Builder —————
function buildExportJSON(record) {
    return JSON.stringify({
        fge_registry_version: "2.0",
        id: record.id,
        character: record.character,
        sessionId: record.sessionId,
        class: record.class,
        canonVersion: record.canonVersion,
        date: record.date,
        status: record.status,
        laneStatus: record.laneStatus,
        notes: record.notes,
        fingerprint: record.fingerprint,
        deltaLog: record.deltaLog,
        authenticationV1: record.authenticationV1,
        _meta: {
            exportedAt: new Date().toISOString(),
            skillTarget: "/home/workdir/.grok/skills/fge-canon-session-manager/",
            registryPath: `/root/.grok/skills/fge-canon-engine/registry/${record.id}_${
            system: "FGE Canon Engine v2.1",
        }
    }, null, 2);
}

```

```

// — Shortcut Spec Builder —————
function buildShortcutSpec(record) {
    return `# FGE RELEASE GATE · iOS Shortcut Spec
# For: ${record.character} · ${record.id}
# _____

```

SHORTCUT NAME: FGE Release Gate

ACTIONS (exact order):

1. SELECT PHOTOS
 - Limit: 1

Select Multiple: OFF

2. GET DETAILS OF PHOTOS

Get: Filename, Date Taken, Dimensions, File Size

3. TEXT — structured payload template:

```
{
  "photoLibraryRef": "[[Filename]]",
  "dateTaken": "[[Date Taken]]",
  "dimensions": "[[Dimensions]]",
  "character": "${record.character}",
  "canonVersion": "${record.canonVersion}",
  "sessionId": "${record.sessionId}",
  "fingerprint": "${record.fingerprint}",
  "confirmedBy": "shortcut",
  "timestamp": "[[Current Date]]"
}
```

4. COPY TO CLIPBOARD

5. SHOW RESULT:

"FGE Release Gate payload copied.
Open SuperGrok → paste for Auth v1."

USAGE (under 30 seconds):

Run shortcut → pick exact photo from release
library → payload hits clipboard → paste into
Grok session at Authentication v1 gate.

Skill processes → scores match → writes
authenticationV1 block into close document.

Drop close document into registry →
flip lockedForRelease: true on confirmed pass.

NOTHING reaches Gumroad without lockedForRelease: true

```
`;  
}
```

// — Default Auth Block —

```
const defaultAuth = () => ({  
  status: "pending",  
  matchScore: 0.00,  
  photoLibraryRef: "",  
  confirmedBy: "",  
  lockedForRelease: false,
```

```
    lockTimestamp: null,  
  });  
};
```

```
// — Initial Records —————
```

```
const INITIAL_RECORDS = [  
  {  
    id: 'FGV-001', character: 'ISOLDE',  
    sessionId: 'SB001-ISO-2026-001', class: 'Canon',  
    canonVersion: '1.0', date: '2026-05-18T00:00:00',  
    status: 'prerelease',  
    laneStatus: { 11: true, 12: true, 13: false, 14: false },  
    notes: 'Initial seed drop. Lead character for Gumroad launch.',  
    deltaLog: [],  
    authenticationV1: defaultAuth(),  
  },  
  {  
    id: 'FGV-002', character: 'RAVEN SB_330',  
    sessionId: 'SB330-RAV-2025-118', class: 'Artifact',  
    canonVersion: '1.2', date: '2025-05-18T00:00:00',  
    status: 'released',  
    laneStatus: { 11: true, 12: true, 13: true, 14: false },  
    notes: 'Competing lanes detected post-session. Unreproducible configuration.  
    deltaLog: [  
      { canonVersion: '1.1', date: '2025-06-01T00:00:00', note: 'Lane 2/Lane 1 se  
      { canonVersion: '1.2', date: '2025-08-15T00:00:00', note: 'Gain threshold m  
    ],  
    authenticationV1: {  
      status: 'confirmed', matchScore: 0.94,  
      photoLibraryRef: 'IMG_3302.JPG',  
      confirmedBy: 'visual', lockedForRelease: true,  
      lockTimestamp: '2025-05-20T14:32:00Z',  
    },  
  },  
  {  
    id: 'FGV-003', character: 'CALISTA VOSS',  
    sessionId: 'SB200-CAL-2026-044', class: 'Canon',  
    canonVersion: '1.0', date: '2026-03-22T00:00:00',  
    status: 'released',  
    laneStatus: { 11: true, 12: true, 13: true, 14: false },  
    notes: 'Clean lane separation. Canon lock achieved at 0.96.',  
    deltaLog: [],  
    authenticationV1: {  
      status: 'confirmed', matchScore: 0.97,  
      photoLibraryRef: 'IMG_4501.JPG',  
      confirmedBy: 'shortcut', lockedForRelease: true,  
      lockTimestamp: '2026-03-23T09:14:00Z',  
    },  
  },  
];
```

```

    },
].map(r => ({ ...r, fingerprint: generateFingerprint(`${r.character}${r.sessionId}`) }));

const LANE_LABELS = {
  11: 'L1 · Identity Anchor',
  12: 'L2 · Gain / State',
  13: 'L3 · Scene / Environment',
  14: 'L4 · Escalation Vector',
};

// — Paste Auth Parser —————
function parseCloseDocument(text) {
  try {
    const json = JSON.parse(text);
    const a = json.authenticationV1 || {};
    return {
      status: a.status === 'PASSED' ? 'confirmed' : a.status === 'FAILED' ? 'fail' : 'unknown',
      matchScore: parseFloat(a.matchScore) || 0,
      photoLibraryRef: a.referenceSource || a.photoLibraryRef || '',
      confirmedBy: a.confirmedBy || 'descriptor',
      lockedForRelease: false,
      lockTimestamp: null,
    };
  } catch { return null; }
}

// — Main App —————
export default function FGEVaultRegistry() {
  const [records, setRecords] = useState(INITIAL_RECORDS);
  const [selected, setSelected] = useState(INITIAL_RECORDS[0]);
  const [tab, setTab] = useState('registry');
  const [subTab, setSubTab] = useState('detail');
  const [filter, setFilter] = useState('all');
  const [boot, setBoot] = useState(false);
  const [tick, setTick] = useState(0);
  const [copied, setCopied] = useState('');
  const [pasteInput, setPasteInput] = useState('');
  const [parseError, setParseError] = useState('');

  const [form, setForm] = useState({
    character: '', sessionId: '', class: 'Canon',
    canonVersion: '1.0', notes: '',
    laneStatus: { 11: false, 12: false, 13: false, 14: false },
  });

  useEffect(() => {
    setTimeout(() => setBoot(true), 300);
  });
}

```

```

    const iv = setInterval(() => setTick(t => t + 1), 1000);
    return () => clearInterval(iv);
}, []);

const syncSelected = (updatedRecord) => {
  setRecords(prev => prev.map(r => r.id === updatedRecord.id ? updatedRecord :
    setSelected(updatedRecord);
});

const copyText = (text, label) => {
  navigator.clipboard?.writeText(text).catch(() => {});
  setCopied(label);
  setTimeout(() => setCopied(''), 2000);
};

const handleExport = () => copyText(buildExportJSON(selected), 'json');
const handleShortcutCopy = () => copyText(buildShortcutSpec(selected), 'shortcu

const handleLockRelease = () => {
  if (selected.authenticationV1.status !== 'confirmed') return;
  const updated = {
    ...selected,
    authenticationV1: {
      ...selected.authenticationV1,
      lockedForRelease: true,
      lockTimestamp: new Date().toISOString(),
    },
  };
  syncSelected(updated);
};

const handlePasteClose = () => {
  setParseError('');
  const parsed = parseCloseDocument(pasteInput);
  if (!parsed) { setParseError('Could not parse close document. Paste valid JSO
  const updated = { ...selected, authenticationV1: { ...selected.authentication
  syncSelected(updated);
  setPasteInput('');
};

const handleAuthFieldUpdate = (field, value) => {
  const updated = {
    ...selected,
    authenticationV1: { ...selected.authenticationV1, [field]: value },
  };
  syncSelected(updated);
};

```

```

const handleNewRecord = () => {
  if (!form.character || !form.sessionId) return;
  const now = new Date().toISOString();
  const rec = {
    id: `FGV-${String(records.length + 1).padStart(3, '0')}`,
    character: form.character.toUpperCase(),
    sessionId: form.sessionId,
    class: form.class,
    canonVersion: form.canonVersion,
    date: now,
    status: 'prerelease',
    laneStatus: { ...form.laneStatus },
    notes: form.notes,
    deltaLog: [],
    authenticationV1: defaultAuth(),
    fingerprint: generateFingerprint(`${form.character}${form.sessionId}${now}`
  );
  setRecords(prev => [rec, ...prev]);
  setSelected(rec);
  setTab('registry');
  setSubTab('detail');
  setForm({ character: '', sessionId: '', class: 'Canon', canonVersion: '1.0',
};

const filtered = records.filter(r => filter === 'all' || r.class.toLowerCase()
const isArtifact = selected?.class === 'Artifact';
const cc = isArtifact ? C.gold : C.accent;
const auth = selected?.authenticationV1 || defaultAuth();
const releaseReady = auth.status === 'confirmed' && !auth.lockedForRelease;
const locked = auth.lockedForRelease;

const s = (base, extra = {}) => ({ fontFamily: '"Courier New',monospace", ...ba

return (
  <div style={{ background: C.void, minHeight: '100vh', color: C.pearl, display

    /* Top classification band */
    <div style={{ height: 4, background: `repeating-linear-gradient(90deg,${C.r

    /* Header */
    <div style={{ background: C.deep, borderBottom: `1px solid ${C.chromeDim}`,
      <div>
        <div style={s({ fontSize: 8, color: C.gold, letterSpacing: 4, marginBot
          <div style={s({ fontSize: 20, color: C.white, letterSpacing: 3, fontWei
            <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing: 3, marg
          </div>
        </div>
      </div>
    </div>

```



```

<div style={{ textAlign: 'right' }}>
  <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing: 2, marg
    <div style={s({ fontSize: 11, color: C.green, letterSpacing: 2 })}>● SY
    <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing: 2, marg
  </div>
</div>

{/* Tab Bar */}
<div style={{ display: 'flex', background: C.deep, borderBottom: `1px solid
  {[{ k: 'registry', l: 'REGISTRY' }, { k: 'new', l: '+ NEW RECORD' }, { k:
    <button key={t.k} onClick={() => setTab(t.k)} style={s({
      background: 'none', border: 'none',
      borderBottom: tab === t.k ? `2px solid ${C.accent}` : '2px solid tran
      color: tab === t.k ? C.white : C.chromeDim,
      fontSize: 8, letterSpacing: 3, padding: '11px 18px', cursor: 'pointer
    })}>{t.l}</button>
  )}}
<div style={{ flex: 1 }} />
<div style={{ display: 'flex', alignItems: 'center', gap: 6, padding: '0
  [['all', 'canon', 'artifact']].map(f => (
    <button key={f} onClick={() => setFilter(f)} style={s({
      background: filter === f ? (f === 'artifact' ? C.goldDim : f === 'c
      border: `1px solid ${filter === f ? (f === 'artifact' ? C.gold : f
      color: filter === f ? (f === 'artifact' ? C.gold : f === 'canon' ?
      fontSize: 7, letterSpacing: 2, padding: '4px 10px', cursor: 'pointe
      borderRadius: 2, fontStyle: f !== 'all' ? 'italic' : 'normal',
    })}>{f.toUpperCase()}</button>
  )}}
</div>
</div>

<div style={{ flex: 1, display: 'flex', overflow: 'hidden' }}>

  {/* — Left: Record List — */}
  {tab !== 'new' && tab !== 'shortcut' && (
    <div style={{ width: 290, borderRight: `1px solid ${C.chromeDim}`, over
      <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing: 2, ma
        {filtered.length} RECORDS · {records.filter(r => r.authenticationV1
      </div>
      {filtered.map(r => {
        const rAuth = r.authenticationV1 || defaultAuth();
        const rcc = r.class === 'Artifact' ? C.gold : C.accent;
        const isSel = selected?.id === r.id;
        return (
          <div key={r.id} onClick={() => { setSelected(r); setSubTab('detai
            style={{
              background: isSel ? (r.class === 'Artifact' ? C.goldFaint : C

```

```

border: `1px solid ${isSel ? rcc : C.chromeDim}`,
borderRadius: 4, padding: '10px 12px', cursor: 'pointer',
transition: 'all 0.2s', marginBottom: 5,
}}>
<div style={{ display: 'flex', justifyContent: 'space-between',
  <span style={s({ fontSize: 7, color: rcc, fontStyle: 'italic'
    <div style={{ display: 'flex', gap: 4, alignItems: 'center' }
      {rAuth.lockedForRelease && <span style={{ fontSize: 9 }}><img alt="lock icon" data-bbox="965 150 985 165"/>
    <div style={{ width: 6, height: 6, borderRadius: '50%', bac
      </div>
    </div>
    <div style={s({ fontSize: 11, color: C.pearl, letterSpacing: 1,
      <div style={s({ fontSize: 7, color: C.chrome, letterSpacing: 1
    </div>
  )};
  }}}
</div>
)}

{/* — Right Panel — */}
<div style={{ flex: 1, overflow: 'auto', padding: 22 }}>

  {/* — NEW RECORD — */}
  {tab === 'new' && (
    <div style={{ maxWidth: 580 }}>
      <div style={s({ fontSize: 9, color: C.gold, letterSpacing: 4, margi

    <div style={{ display: 'flex', gap: 10, marginBottom: 20 }}>
      {[ 'Canon', 'Artifact' ].map(cls => (
        <button key={cls} onClick={() => setForm(f => ({ ...f, class: c
          background: form.class === cls ? (cls === 'Artifact' ? C.gold
          border: `1px solid ${form.class === cls ? (cls === 'Artifact'
          color: form.class === cls ? (cls === 'Artifact' ? C.gold : C.
          fontSize: 10, letterSpacing: 3, padding: '10px 24px', cursor:
          fontStyle: 'italic', borderRadius: 3, transition: 'all 0.2s',
        )}}>{cls}</button>
      )}}
    </div>

    [[
      { k: 'character', l: 'CHARACTER NAME', p: 'ISOLDE · RAVEN SB_330.
      { k: 'sessionId', l: 'SESSION ID', p: 'SB001-ISO-2026-001' },
      { k: 'canonVersion', l: 'CANON VERSION', p: '1.0' },
    ].map(f => (
      <div key={f.k} style={{ marginBottom: 14 }}>
        <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing:
          <input value={form[f.k]} onChange={e => setForm(p => ({ ...p, [

```

```

        placeholder={f.p} style={s({
            width: '100%', background: C.titanium, border: `1px solid $
            borderRadius: 3, padding: '9px 12px', color: C.pearl,
            fontSize: 10, letterSpacing: 1, outline: 'none', boxSizing:
        })} />
    </div>
    )))

<div style={{ marginBottom: 18 }}>
    <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing: 2
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', ga
        (Object.entries(LANE_LABELS).map(([k, l]) => (
            <div key={k} onClick={() => setForm(f => ({ ...f, laneStatus:
                style={{
                    display: 'flex', alignItems: 'center', gap: 8,
                    background: form.laneStatus[k] ? C.greenFaint : C.titaniu
                    border: `1px solid ${form.laneStatus[k] ? C.green : C.chr
                    borderRadius: 3, padding: '8px 12px', cursor: 'pointer',
                }}>
            <div style={{ width: 7, height: 7, borderRadius: '50%', bac
            <span style={s({ fontSize: 7, color: form.laneStatus[k] ? C
            </div>
        ))}
    </div>
</div>

<div style={{ marginBottom: 22 }}>
    <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing: 2
    <textarea value={form.notes} onChange={e => setForm(f => ({ ...f,
        placeholder="Configuration notes, gain ceiling, competing lanes
        rows={4} style={s({
            width: '100%', background: C.titanium, border: `1px solid ${C
            borderRadius: 3, padding: '9px 12px', color: C.pearl,
            fontSize: 10, letterSpacing: 1, outline: 'none', resize: 'ver
        })} />
    </div>

    <button onClick={handleNewRecord} style={s({
        background: C.gold, border: 'none', borderRadius: 3,
        padding: '11px 28px', color: C.void, fontSize: 9, letterSpacing:
    })}>REGISTER · PASS QUALITY GATE</button>
</div>
))

{ /* — SHORTCUT SPEC — */ }
{tab === 'shortcut' && selected && (
    <div style={{ maxWidth: 620 }}>

```

```

<div style={s({ fontSize: 9, color: C.gold, letterSpacing: 4, margi
<div style={s({ fontSize: 8, color: C.chrome, letterSpacing: 1, mar
  For: {selected.character} · {selected.id} · {selected.fingerprint
</div>

<div style={{
  background: C.titanium, border: `1px solid ${C.chromeDim}`,
  borderRadius: 4, padding: 18, marginBottom: 16,
}}>
  <pre style={s({ fontSize: 9, color: C.pearl, lineHeight: 1.7, mar
    {buildShortcutSpec(selected)}
  </pre>
</div>

<button onClick={handleShortcutCopy} style={s({
  background: copied === 'shortcut' ? C.green : C.gold,
  border: 'none', borderRadius: 3, padding: '10px 24px',
  color: C.void, fontSize: 9, letterSpacing: 3, cursor: 'pointer',
  transition: 'background 0.3s',
}})}>
  {copied === 'shortcut' ? '✓ COPIED' : 'COPY SHORTCUT SPEC'}
</button>

<div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing: 2,
  BUILD IN: Settings → Shortcuts → + → add actions in order above.<
  Pre-fill character/canonVersion/sessionId/fingerprint fields.<br
  Use "Insert Variable" for [[ ]] dynamic fields.<br />
  Run time: under 30 seconds. Auth strength: ~0.97
</div>
</div>
)}

{/* — RECORD DETAIL — */}
{tab === 'registry' && selected && (
  <div>
    {/* Sub-tabs */}
    <div style={{ display: 'flex', gap: 0, marginBottom: 20, borderBott
      [{ k: 'detail', l: 'RECORD' }, { k: 'auth', l: 'AUTH v1 GATE' }],
      <button key={t.k} onClick={() => setSubTab(t.k)} style={s({
        background: 'none', border: 'none',
        borderBottom: subTab === t.k ? `2px solid ${cc}` : '2px solid
        color: subTab === t.k ? C.white : C.chromeDim,
        fontSize: 8, letterSpacing: 3, padding: '8px 16px', cursor: '
        ...(t.k === 'auth' && auth.status === 'pending' ? { color: C.
      })}>{t.l}{t.k === 'auth' && auth.status === 'pending' ? ' ⌚ ':
      )}}
    </div>
  )}

```

```

{ /* — DETAIL SUB-TAB — */}
{subTab === 'detail' && (
  <div>
    <div style={{ display: 'flex', justifyContent: 'space-between',
      <div>
        <div style={s({ fontSize: 9, color: cc, letterSpacing: 4, f
          <div style={s({ fontSize: 22, color: C.white, letterSpacing
            <div style={s({ fontSize: 8, color: C.chrome, letterSpacing
              <AuthBadge auth={auth} />
            </div>
          </div>
        <div style={{ textAlign: 'center' }}>
          <QRDisplay data={JSON.stringify({ id: selected.id, fp: sele
            isArtifact={isArtifact} sessionId={selected.sessionId} si
          <div style={s({ fontSize: 6, color: cc, letterSpacing: 2, m
            </div>
          </div>
        </div>

    { /* Fingerprint */}
    <div style={{ background: C.titanium, border: `1px solid ${C.ch
      <span style={s({ fontSize: 7, color: C.chromeDim, letterSpaci
        <span style={s({ fontSize: 13, color: cc, letterSpacing: 4 })
      </div>

    { /* Lane Status */}
    <div style={{ marginBottom: 16 }}>
      <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacin
        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(4
          {Object.entries(LANE_LABELS).map(([k, l]) => {
            const on = selected.laneStatus[k];
            return (
              <div key={k} style={{ background: on ? C.greenFaint : C
                <div style={{ width: 7, height: 7, borderRadius: '50%
                  <div style={s({ fontSize: 6, color: on ? C.green : C.
                </div>
              </div>
            );
          })}
        </div>
      </div>

    { /* Status */}
    <div style={{ marginBottom: 16 }}>
      <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacin
        <div style={{ display: 'flex', gap: 7 }}>
          {['prerelease', 'released', 'archived'].map(st => (
            <button key={st} onClick={() => syncSelected({ ...selecte
              background: selected.status === st ? (st === 'released'

```

```

        border: `1px solid ${selected.status === st ? (st === '
        color: selected.status === st ? (st === 'released' ? C.
        fontSize: 7, letterSpacing: 2, padding: '6px 13px', cur
    )}>{st.toUpperCase()}</button>
    )))
</div>
</div>

{/* Notes */}
<div style={{ marginBottom: 16 }}>
    <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing:
    <div style={{ background: C.titanium, border: `1px solid ${C.
        <div style={s({ fontSize: 9, color: C.chrome, letterSpacing
    </div>
</div>

{/* Delta Log */}
{isArtifact && selected.deltaLog.length > 0 && (
    <div style={{ marginBottom: 16 }}>
        <div style={s({ fontSize: 7, color: C.gold, letterSpacing:
            CANON DELTA LOG · {selected.deltaLog.length} ENTRIES · AP
        </div>
        {selected.deltaLog.map((d, i) => (
            <div key={i} style={{ borderLeft: `2px solid ${C.goldDim}
                <div style={s({ fontSize: 8, color: C.gold, letterSpacing:
                    <div style={s({ fontSize: 9, color: C.chrome, letterSpacing:
                </div>
            ))}
        </div>
    )}
</div>
)}
)}
)}

{/* — AUTH v1 GATE SUB-TAB — */}
{subTab === 'auth' && (
    <div>
        <div style={s({ fontSize: 9, color: C.gold, letterSpacing: 4, m
        <div style={s({ fontSize: 8, color: C.chromeDim, letterSpacing:
            This gate must pass before any card reaches Gumroad.<br />
            lockedForRelease: true is the single source of release truth.
        </div>

    </div>

    <div style={{ background: C.titanium, border: `1px solid ${auth
        <div style={{ display: 'flex', justifyContent: 'space-between
            <AuthBadge auth={auth} />
            {auth.matchScore > 0 && (

```

```

        <div style={s({ fontSize: 11, color: auth.matchScore >= 0
            {(auth.matchScore * 100).toFixed(0)}% MATCH
        </div>
    )}
</div>
<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr'
    {
        { 1: 'PHOTO LIBRARY REF', v: auth.photoLibraryRef || '-'
        { 1: 'CONFIRMED BY', v: auth.confirmedBy || '-' },
        { 1: 'LOCK TIMESTAMP', v: auth.lockTimestamp ? new Date(a
        { 1: 'FINGERPRINT', v: selected.fingerprint },
    ].map(f => (
        <div key={f.1}>
            <div style={s({ fontSize: 6, color: C.chromeDim, letter
            <div style={s({ fontSize: 8, color: C.pearl, letterSpac
            </div>
        )}}
    </div>
</div>

{/* Manual Auth Fields */}
<div style={{ marginBottom: 18 }}>
    <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacin
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr'
        <div>
            <div style={s({ fontSize: 6, color: C.chromeDim, letterSp
            <select value={auth.status} onChange={e => handleAuthFiel
                style={s({ width: '100%', background: C.titanium, borde
                <option value="pending">pending</option>
                <option value="confirmed">confirmed</option>
                <option value="failed">failed</option>
            </select>
        </div>
        <div>
            <div style={s({ fontSize: 6, color: C.chromeDim, letterSp
            <input type="number" min="0" max="1" step="0.01"
                value={auth.matchScore}
                onChange={e => handleAuthFieldUpdate('matchScore', pars
                style={s({ width: '100%', background: C.titanium, borde
            </div>
        </div>
    <div style={{ marginBottom: 10 }}>
        <div style={s({ fontSize: 6, color: C.chromeDim, letterSpac
        <input value={auth.photoLibraryRef}
            onChange={e => handleAuthFieldUpdate('photoLibraryRef', e
            placeholder="IMG_XXXX.JPG or shortcut payload filename"
            style={s({ width: '100%', background: C.titanium, border:

```

```

</div>
<div style={{ marginBottom: 10 }}>
  <div style={s({ fontSize: 6, color: C.chromeDim, letterSpacing: 0.5 })}>
    <select value={auth.confirmedBy} onChange={e => handleAuthF
      style={s({ width: '100%', background: C.titanium, border:
        <option value="">— select —</option>
        <option value="visual">visual</option>
        <option value="descriptor">descriptor</option>
        <option value="shortcut">shortcut</option>
      </select>
    </div>
  </div>
</div>

```

```

{/* Paste Close Document */}
<div style={{ marginBottom: 18 }}>
  <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing: 0.5 })}>
    <textarea value={pasteInput} onChange={e => setPasteInput(e.t
      placeholder='Paste JSON close document from Grok session he
      rows={5} style={s({ width: '100%', background: C.titanium,
        {parseError && <div style={s({ fontSize: 7, color: C.red, let
        <button onClick={handlePasteClose} style={s({ marginTop: 8, b
          PARSE · APPLY AUTH BLOCK
        </button>
      </div>
    </div>
  </div>

```

```

{/* Lock for Release */}
<div style={{ background: locked ? C.greenFaint : releaseReady
  <div style={s({ fontSize: 8, color: locked ? C.green : releas
    {locked ? '🔒 LOCKED FOR RELEASE — GUMROAD ELIGIBLE' : rele
  </div>
  {!locked && (
    <button onClick={handleLockRelease} disabled={!releaseReady
      background: releaseReady ? C.gold : C.chromeDim,
      border: 'none', borderRadius: 3, padding: '10px 24px',
      color: releaseReady ? C.void : C.deep, fontSize: 9, lette
      cursor: releaseReady ? 'pointer' : 'not-allowed', fontWei
      opacity: releaseReady ? 1 : 0.5,
    </button>
  )}
  {locked && (
    <div style={s({ fontSize: 8, color: C.green, letterSpacing:
      Locked at: {auth.lockTimestamp ? new Date(auth.lockTimest
    </div>
  )}
</div>

```



```

    </div>
  )}

  {/* — EXPORT SUB-TAB — */}
  {subTab === 'export' && (
    <div>
      <div style={s({ fontSize: 9, color: C.gold, letterSpacing: 4, m
      <div style={s({ fontSize: 8, color: C.chromeDim, letterSpacing:
        Copy this JSON and drop into:<br />
        <span style={{ color: C.accent }}>/home/workdir/.grok/skills/
      </div>

      <div style={{ background: C.titanium, border: `1px solid ${C.ch
        <pre style={s({ fontSize: 8, color: C.pearl, lineHeight: 1.6,
          {buildExportJSON(selected)}
        </pre>
      </div>

      <div style={{ display: 'flex', gap: 10 }}>
        <button onClick={handleExport} style={s({
          background: copied === 'json' ? C.green : C.gold,
          border: 'none', borderRadius: 3, padding: '10px 22px',
          color: C.void, fontSize: 9, letterSpacing: 3, cursor: 'poin
        })}>{copied === 'json' ? '✓ COPIED' : 'COPY JSON · EXPORT TO

        <button onClick={() => { setTab('shortcut'); }} style={s({
          background: C.titanium, border: `1px solid ${C.chromeDim}`,
          borderRadius: 3, padding: '10px 22px',
          color: C.chrome, fontSize: 9, letterSpacing: 3, cursor: 'po
        })}>VIEW SHORTCUT SPEC →</button>
      </div>

      <div style={s({ fontSize: 7, color: C.chromeDim, letterSpacing:
        Filename convention: {selected.id}_${selected.character.replac
        Skill reads this file at session open → full context loaded a
        authenticationV1.lockedForRelease must be true before Gumroad
      </div>
    </div>
  )}
</div>
)}
</div>

  {/* Bottom band */}
  <div style={{ height: 4, background: `repeating-linear-gradient(90deg,${C.r
</div>

```

